

Отчёт по лабораторной работе №14

**Программирование в командном процессоре ОС UNIX.
Расширенное программирование**

Юсупова Амина Руслановна

Содержание

1	Цель работы	5
2	Теоретические сведения	6
2.1	Команда <code>getopts</code>	6
3	Выполнение лабораторной работы	8
3.1	Задание 1	8
3.2	Задание 2	9
3.3	Задание 3	10
4	Выводы	12
5	Ответы на контрольные вопросы	13
5.1	1. Найдите синтаксическую ошибку в следующей строке:	13
5.2	2. Как объединить (конкатенация) несколько строк в одну?	13
5.3	3. Найдите информацию об утилите <code>seq</code> . Какими иными способами можно реализовать её функционал при программировании на <code>bash</code> ?	14
5.4	4. Какой результат даст вычисление выражения $((10 / 3))$?	15
5.5	5. Укажите кратко основные отличия командной оболочки <code>zsh</code> от <code>bash</code>	16
5.6	6. Проверьте, верен ли синтаксис данной конструкции	16
5.7	7. Сравните язык <code>bash</code> с какими-либо языками программирования. Какие преимущества у <code>bash</code> по сравнению с ними? Какие недостатки?	17

Список иллюстраций

3.1	Скрипт	8
3.2	Исполняемый файл	9
3.3	Скрипт	9
3.4	Запуск	9
3.5	Запуск	10
3.6	Скрипт	11
3.7	Исполняемый файл	11

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Теоретические сведения

2.1 Команда getopt

Команда `getopts` используется в `bash` для разбора аргументов командной строки. Она позволяет обрабатывать опции (ключи) и их аргументы, что делает скрипты более гибкими и удобными для пользователя. Синтаксис:

```
while getopts "опции" переменная; do
    case $переменная in опция) действия ;;
    esac
done
```

Управляющие конструкции `bash`

Конструкция	Назначение
if-then-else	Условное выполнение команд
for	Цикл с перебором списка значений
while	Цикл с предусловием
until	Цикл с постусловием
case	Множественное ветвление

Команды `break` и `continue` - `break` — прерывает выполнение цикла

- `continue` — прерывает текущую итерацию и переходит к следующей

Команды true и false - true — всегда возвращает код завершения 0 (истина)

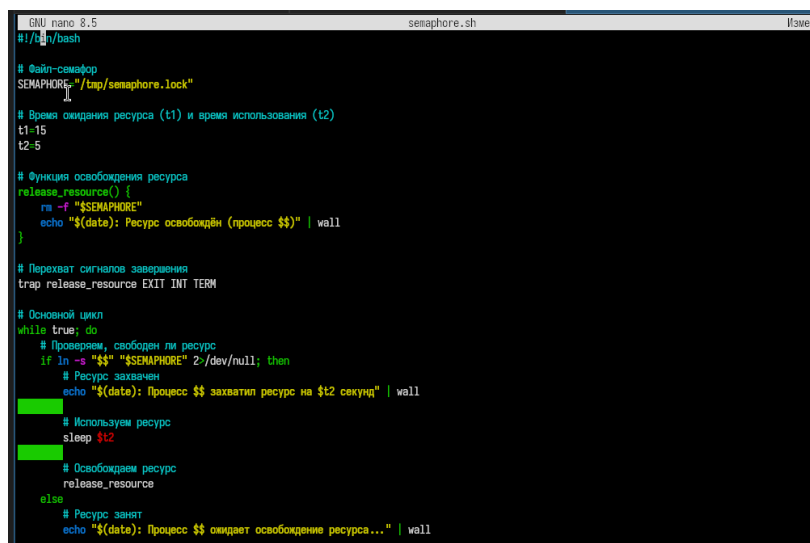
- false — всегда возвращает код завершения 1 (ложь)

3 Выполнение лабораторной работы

3.1 Задание 1

3.1.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `semaphore.sh`. А далее я написала скрипт, который при запуске будет работать по механизму семафоров



```
GNU nano 8.5 semaphore.sh
#!/bin/bash

# Файл-семафор
SEMAPHORE="/tmp/semaphore.lock"

# Время ожидания ресурса (t1) и время использования (t2)
t1=15
t2=5

# Функция освобождения ресурса
release_resource() {
    rm -f "$SEMAPHORE"
    echo "$(date): Ресурс освобожден (процесс $$)" | wall
}

# Перехват сигналов завершения
trap release_resource EXIT INT TERM

# Основной цикл
while true; do
    # Проверим, свободен ли ресурс
    if ln -s "$SEMAPHORE" 2>/dev/null; then
        # Ресурс захвачен
        echo "$(date): Процесс $$ захватил ресурс на $t2 секунд" | wall
        # Используем ресурс
        sleep $t2
        # Освобождаем ресурс
        release_resource
    else
        # Ресурс занят
        echo "$(date): Процесс $$ ожидает освобождение ресурса..." | wall
    fi
done
```

Рисунок 3.1: Скрипт

3.1.2 2. Исполняемый скрипт и запуск

Теперь я делаю скрипт исполняемым, чтобы программа работала. И запускаю его


```

aryusupova@aryusupova:~$ chmod +x semaphore.sh
aryusupova@aryusupova:~$ ./semaphore.sh > /dev/tty2 &
[1] 3374
aryusupova@aryusupova:~$ ./semaphore.sh

```

Рисунок 3.2: Исполняемый файл

3.2 Задание 2

3.2.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `my_man.sh`. А далее я написала скрипт, который является аналогом команды `man` в терминале.

```

GNU nano 8.5 my_man.sh
#!/bin/bash

# Проверка наличия аргумента
if [ -z "$1" ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

COMMAND="$1"
MAN_DIR="/usr/share/man/man1"

# Поиск файла справки
MAN_FILE=$(find "$MAN_DIR" -name "${COMMAND}.*" 2>/dev/null | head -1)

if [ -n "$MAN_FILE" ]; then
    # Открываем справку
    if command -v less &>/dev/null; then
        zcat "$MAN_FILE" | less
    else
        zcat "$MAN_FILE"
    fi
else
    echo "Справка по команде '$COMMAND' не найдена"
    exit 1
fi

```

Рисунок 3.3: Скрипт

3.2.2 2. Запуск скрипта

Сначала делаю файл исполняемым и потом запускаю

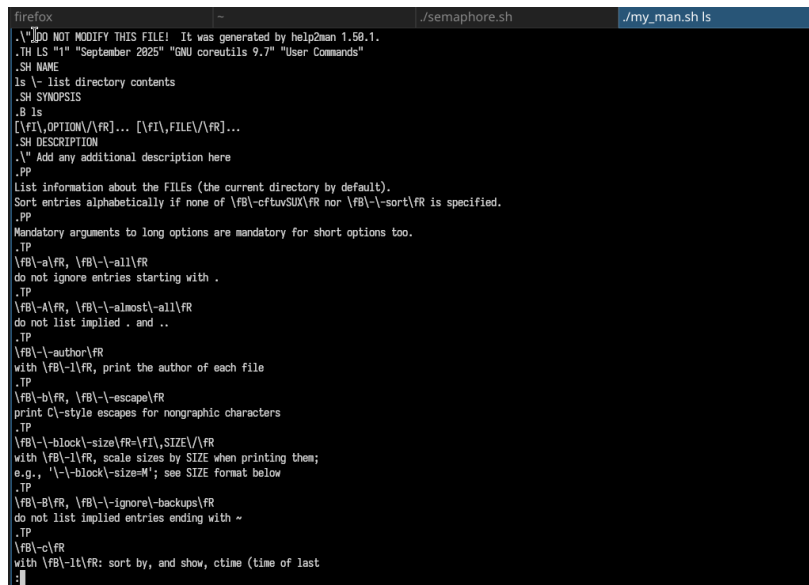
```

aryusupova@aryusupova:~$ chmod +x my_man.sh
aryusupova@aryusupova:~$ ./my_man.sh ls

```

Рисунок 3.4: Запуск

Вы можете увидеть результат запуска программы



```
firefox ~ ./semaphore.sh ./my_man.sh ls
.\"DO NOT MODIFY THIS FILE! It was generated by help2man 1.50.1.
.TH LS "1" "September 2025" "GNU coreutils 9.7" "User Commands"
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[(Ff)\OPTION\...\] [(fF)\FILE\...\]
.SH DESCRIPTION
.\" Add any additional description here
.PP
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of \FB\~cftuvSX\FR nor \FB\~\~sort\FR is specified.
.PP
Mandatory arguments to long options are mandatory for short options too.
.TP
\FB\~a\FR, \FB\~\~all\FR
do not ignore entries starting with .
.TP
\FB\~A\FR, \FB\~\~almost\~all\FR
do not list implied . and ..
.TP
\FB\~\~author\FR
with \FB\~l\FR, print the author of each file
.TP
\FB\~b\FR, \FB\~\~escape\FR
print C\~style escapes for nongraphic characters
.TP
\FB\~\~block\~size\FR=(Ff)\SIZE\FR
with \FB\~l\FR, scale sizes by SIZE when printing them;
e.g., '\~\~block\~size=M'; see SIZE format below
.TP
\FB\~B\FR, \FB\~\~ignore\~backups\FR
do not list implied entries ending with ~
.TP
\FB\~c\FR
with \FB\~lt\FR: sort by, and show, ctime (time of last
```

Рисунок 3.5: Запуск

3.3 Задание 3

3.3.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `random_letters.sh`. А далее я написала скрипт, который на вход получает количество символов, а далее из рандомных букв составляет слово.

```

GNU nano 8.5                                random_letters.sh
#!/bin/bash

# Длина последовательности (по умолчанию 20)
LENGTH=${1:-20}
ALPHABET="abcdefghijklmnopqrstuvwxyz"
ALPH_LEN=26

# Генерация случайной последовательности
for ((i=0; i<$LENGTH; i++)); do
    RAND_NUM=$((RANDOM % ALPH_LEN))
    echo -n "${ALPHABET:$RAND_NUM:1}"
done
echo ""

```

Рисунок 3.6: Скрипт

3.3.2 2. Исполняемый скрипт и запуск

Теперь я делаю скрипт исполняемым и запускаю

```

aryusupova@aryusupova:~$ chmod +x random_letters.sh
aryusupova@aryusupova:~$ ./random_letters.sh 30
qobrcgxjgowstf1bbqwfgqmooqdkp
aryusupova@aryusupova:~$ |

```

Рисунок 3.7: Исполняемый файл

4 Выводы

В ходе выполнения лабораторной работы 14 я изучила основы программирования в оболочке ОС UNIX/Linux и получила практические навыки написания более сложных командных файлов с использованием логических управляющих конструкций, циклов и механизмов взаимодействия процессов.

5 Ответы на контрольные вопросы

5.1 1. Найдите синтаксическую ошибку в следующей строке:

```
while [\\$1 != "exit"]
```

Ошибки:

- Лишний обратный слеш перед \$1: \ \$1 — вместо этого должно быть «\$1» или \$1
- Отсутствует пробел после [и перед]
- Отсутствует закрывающая квадратная скобка]

Исправленный вариант:

```
while [ "$1" != "exit" ]
```

5.2 2. Как объединить (конкатенация) несколько строк в одну?

Способы конкатенации строк в bash:

```

# Способ 1: простой перебор (самый распространённый)
str1="Hello"
str2="World"
result="$str1 $str2"      # "Hello World"

# Способ 2: использование оператора +=
result=""
for word in Hello World; do
    result+="$word "
done
echo "$result"            # "Hello World "

# Способ 3: использование printf
result=$(printf "%s" "$str1" "$str2")

# Способ 4: удаление символов новой строки
lines=$(cat file.txt)
result="${lines//$'\n'/ }" # замена перевода строки на пробел

```

5.3 3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash?

- Утилита seq — генерирует последовательность чисел.

```

seq 1 5      # 1 2 3 4 5
seq 1 2 10   # 1 3 5 7 9

```

Альтернативные способы без seq:

```
# Способ 1: цикл for (встроенный в bash)
```

```
for i in {1..5}; do
```

```
    echo $i
```

```
done
```

```
# Способ 2: цикл while
```

```
i=1
```

```
while [ $i -le 5 ]; do
```

```
    echo $i
```

```
    ((i++))
```

```
done
```

```
# Способ 3: команда eval
```

```
eval echo {1..5}
```

```
# Способ 4: использование expr
```

```
i=1
```

```
while [ $i -le 5 ]; do
```

```
    echo $i
```

```
    i=$(expr $i + 1)
```

```
done
```

5.4 4. Какой результат даст вычисление выражения

((10 / 3))?

Результат: 3

Пояснение: В bash оператор (()) выполняет целочисленное деление. Остаток отбрасывается.

```
echo $((10 / 3))    # выведет 3
echo $((10 % 3))    # выведет 1 (остаток)
```

5.5 5. Укажите кратко основные отличия командной оболочки zsh от bash.

Характеристика	bash	zsh
Автодополнение	Базовое (Tab)	Расширенное, с подсказками
Массивы	Индексация с 0	Индексация с 1
Глобализация	Стандартная	** (рекурсивный поиск)
Проверка орфографии	Нет	Есть (исправление опечаток)
Промпт	Ограниченные возможности	Расширенные настройки
Совместимость	POSIX-совместимая	Не полностью совместима с POSIX
Интерактивность	Меньше возможностей	Больше возможностей

5.6 6. Проверьте, верен ли синтаксис данной конструкции

```
for ((a=1; a <= LIMIT; a++))
```

Ошибка: Отсутствует закрывающая двойная скобка))

Исправленный вариант:


```
for ((a=1; a <= LIMIT; a++))
```

```
#                                     ^^ должны быть две закрывающие скобки
```

```
# В коде выше показана только одна ) в конце строки
```

Правильно:

```
for ((a=1; a <= LIMIT; a++)); do
```

```
    echo $a
```

```
done
```

5.7 7. Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки?

Сравнение с Python

Характеристика	bash	Python
Предназначение	Управление системой, автоматизация	Универсальное программирование
Синтаксис	Специфичный, много метасимволов	Читаемый, интуитивный
Скорость	Быстрый запуск	Медленнее из-за интерпретации
Типизация	Слабая (всё строки)	Динамическая, строгая

Характеристика	bash	Python
Работа с процессами	Встроенная	Через модули (subprocess)
Массивы	Ограниченные возможности	Полноценные структуры данных

Преимущества bash - Встроенные команды для работы с файлами и процессами

- Лёгкость запуска других программ
- Быстрый старт (нет загрузки интерпретатора)
- Естественность для системного администрирования

Недостатки bash - Сложный синтаксис (пробелы имеют значение, много метасимволов)

- Нет поддержки плавающей запятой
- Нет структур данных (массивы ограничены)
- Сложная отладка
- Медленная обработка больших объёмов текста
- Ограниченные возможности для сложной бизнес-логики